

Uçuş Kontrol Yazılımı

Gömülü gerçek zamanlı yazılımın mimarisi, kaynak kodu ve satır düzeyinde gerekçelendirmesi

«sketch» verici	«sketch» alici	«sketch» arac	«sketch» arac
verici_v2_kumanda.ino el kumandası	alici_v3_ucus_kontrol.ino uçuş kontrolcüsü	arac_esc_kalibrasyon.ino ESC uç nokta	arac_motor_test.ino yön doğrulama

UÇUŞ YAZILIMLARI

ARAÇ / DEVREYE ALMA YAZILIMLARI

Doküman kodu	SWD-QC-002
Üst doküman	SDD-QC-001 — Sistem Tasarım Dokümanı
Dil / araç zinciri	C++ (Arduino framework), ESP32 Arduino Core
Hedef	ESP32-WROOM-32, 240 MHz, çift çekirdek
Kontrol döngüsü	250 Hz sabit periyot
Modül sayısı	2 uçuş yazılımı + 2 araç yazılımı
Toplam satır	≈ 560 satır (yorumlar dâhil)
Bağımlılıklar	RF24, MPU6050_light, ESP32Servo, Wire, SPI

1. Yazılım Mimarisi

Yazılım iki uçuş modülü ve iki araç modülünden oluşur. Uçuş modülleri kalıcı olarak sahada çalışır; araç modülleri yalnızca devreye alma sırasında yüklenir ve doğrulama tamamlandığında kartlarda kalmaz.

Modül	Hedef düğüm	Rol	Yaşam döngüsü
verici_v2_kumanda	Verici ESP32	Pilot girdisini okur, ölçekler ve telsizle yayınlar	Kalıcı
alici_v3_ucus_kontrol	Alıcı ESP32	Komut alır, aç kestirir, PID uygular, motorları sürer	Kalıcı
arac_esc_kalibrasyon	Alıcı ESP32	Dört ESC'ye ortak gaz aralığı öğretir	Bir kerelik
arac_motor_test	Alıcı ESP32	Motorları tek tek çalıştırıp dönüş yönünü doğrular	Devreye alma

1.1 Katmanlı yapı

Uçuş kontrol yazılımı, her katmanın yalnızca bir alttakine bağımlı olduğu düz bir katman yapısıyla kurgulanmıştır. Bu, bir katmanın bağımsız olarak test edilebilmesini sağlar — nitekim geliştirme sırası da tam olarak bu katmanları izlemiştir.



Şekil 1.1 — Yazılım katmanları ve doğrulama sırası. Her katman, bir üsttekine geçilmeden önce bağımsız olarak doğrulanmıştır.

2. Telsiz Haberleşme Protokolü

Haberleşme, sabit uzunlukta bir C yapısının (struct) doğrudan bayt dizisi olarak gönderilmesine dayanır. Bu yaklaşım, serileştirme maliyetini sıfırlar ve gecikmeyi öngörülebilir kılar; karşılığında **iki taraftaki yapı tanımının birebir aynı olması zorunludur**.

2.1 Çerçeve yapısı

Alan	Tip	Bayt	Aralık	Anlam
kanal[0]	uint16_t	0-1	1000-2000	Gaz komutu (µs)
kanal[1]	uint16_t	2-3	1000-2000	Yaw komutu (µs)
kanal[2]	uint16_t	4-5	1000-2000	Pitch komutu (µs)
kanal[3]	uint16_t	6-7	1000-2000	Roll komutu (µs)
sayac	uint16_t	8-9	0-65535	Paket sırası (kayıp tespiti)

Toplam çerçeve boyutu **10 bayt**'tır; nRF24'ün 32 baytlık sabit yük kapasitesinin çok altındadır. 50 Hz çerçeve hızında ham veri yükü yalnızca 4 kbit/s'dir — seçilen 250 kbps veri hızının %2'si. Bu geniş marj, düşük veri hızının menzil avantajından ödün vermeden kullanılabilmesini sağlar.

2.2 Radyo parametreleri ve gerekçeleri

Parametre	Varsayılan	Seçilen	Gerekçe
Veri hızı	1 Mbps	250 kbps	Alıcı duyarlılığı artar, menzil yaklaşık iki katına çıkar; veri yükü zaten çok düşük
Kanal	76 (2476 MHz)	100 (2500 MHz)	2.4 GHz WiFi bandının yoğun bölgesinden uzaklaşma
Çıkış gücü	PA_MAX	PA_LOW (test) / PA_MAX (uçuş)	Modüller çok yakinken yüksek güç alıcıyı doyuma sokabiliyor
Adres	0xE7E7E7E7E7	"DRN01"	Varsayılan adres yaygındır; çakışma riskini azaltmak için özel adres
Otomatik ACK	Açık	Açık	Gönderim başarısı <code>radio.write()</code> dönüş değerinden anlaşılır — bağlantı tespiti için kullanılır
CRC	16 bit	16 bit	Çerçeve bütünlüğü doğrulaması

2.3 Çerçeve doğrulama ve kayıp sayımı

Alıcı, gelen her çerçeveyi işlemeyen önce iki aşamalı bir süzgeçten geçirir. Bu mekanizma, geliştirme sırasında gözlenen gerçek bir arızaya (D-03) yanıt olarak eklenmiştir.

- Aralık süzgeci.** Dört kanalın tümü 950-2050 µs bandında olmalıdır. Bant dışı bir değer, çerçevenin tamamının reddedilmesine yol açar — kısmi kabul yapılmaz, çünkü bozuk bir çerçevede tek bir alanın sağlam olduğu varsayılmaz.
- Sayaç sürekliliği.** Ardışık çerçevelerin sayaç farkı 1'den büyükse aradaki fark kayıp paket olarak birikir. Bu sayaç, telemetri üzerinden menzil ve girişim analizinde kullanılır.

Neden bu süzgeç kritik

Verici beslemesiz kaldığında alıcının SPI tamponu tanımsız içerik döndürür; bu, kanal alanlarında 0 veya 65535 gibi uç değerler olarak görünür. Süzgeç olmasaydı bu değerler doğrudan motor karıştırma matrisine girer ve dört motora aynı anda sıçrama komutu üretirdi. Süzgeç, çerçeve düzeyinde bir *bütünlük sözleşmesi* uygular.

3. Verici Yazılımı — El Kumandası

Verici modülü üç işi sırayla yapar: analog eksenleri okur, bunları standart 1000–2000 μ s bandına ölçekler ve çerçeveyi yayınlar. Basit görünen bu zincirin içinde, gerçek donanımın dayattığı üç tasarım kararı gizlidir.

3.1 Pin tahsisi ve donanım kısıtı

verici_v2_kumanda.ino · satır 17–25

```
17  /* ----- PINLER */
18  #define PIN_NRF_CE  4
19  #define PIN_NRF_CSN  5
20  #define PIN_GAZ      32    // sol joystick - dikey   (ADC1)
21  #define PIN_YAW      33    // sol joystick - yatay   (ADC1)
22  #define PIN_PITCH    34    // sag joystick - dikey   (ADC1)
23  #define PIN_ROLL     35    // sag joystick - yatay   (ADC1)
24  // NOT: Dört eksen de ADC1 pinlerindedir. ADC2 telsizle aynı anda
25  //      kullanılamaz (ESP32 donanım kisiti).
```

Dört eksenin de **ADC1** bloğuna (GPIO 32–39) yerleştirilmesi bir tercih değil, zorunluluktur: ESP32'de ADC2 çevre birimi Wi-Fi/telsiz yığını etkinken kullanılamaz. Ayrıca 38 pinli DevKit sürümü seçilmiştir, çünkü 30 pinli sürüm GPIO 34–39 hatlarını dışarı vermez.

3.2 Kalibrasyon sabitleri

kalibrasyon sabitleri · satır 27–37

```
27  /* ----- KALIBRASYON (OLCUM SONUCU) */
28  // Her joystick serbest birakildiginda okunan ham ADC degeri.
29  // Fabrikasyon toleransi nedeniyle 2048 degil, eksen basina farklidir.
30  const uint16_t MERKEZ_GAZ  = 1850;
31  const uint16_t MERKEZ_YAW  = 1900;
32  const uint16_t MERKEZ_PITCH = 1860;
33  const uint16_t MERKEZ_ROLL  = 1880;
34
35  const uint16_t OLU_BOLGE    = 80;    // +/- ADC birimi - titremeyi susturur
36  const uint16_t GAZ_OLU_BOLGE = 200;  // gaz icin daha genis bant
37  const float    GAZ_HIZI     = 12.0;  // us / dongu - gaz degisim hizi
```

Joystick potansiyometreleri serbest konumda kuramsal orta değer olan 2048'i vermez; ölçülen merkezler eksen başına 1850–1900 bandındadır. Merkez değerleri koda sabit olarak girilmiş ve ölçekleme merkezin iki yanında ayrı ayrı yapılmıştır. Böylece fiziksel kaçıklık ne olursa olsun çıkış tam olarak 1500 μ s'de merkezlenir.

Ölü bölge başlangıçta ± 60 ADC birimi seçilmişti. Seri port kayıtlarında yaw kanalında 1500–1505 arası artık titreşim gözlemlendi; bant ± 80 'e genişletilerek titreşim tümüyle giderildi. Bu, ölçüme dayalı bir düzeltmenin somut örneğidir — titreşim doğrudan motorlara iletilseydi araç yerinde dururken sürekli düzeltme yapmaya çalışırdı.

3.3 Kanal ölçekleme fonksiyonu

kanal0ku() · satır 51–72

```
51  /* =====
52  *   kanal0ku - ham ADC degerini 1000-2000 us bandina cevirir
53  *
54  *   Merkezin iki yaninda ayri olcekleme yapilir. Boylece merkez degeri
55  *   2048'den sapmis olsa bile cikis tam olarak 1500'de merkezlenir.
56  *   ===== */
57  uint16_t kanal0ku(uint8_t pin, uint16_t merkez, bool ters) {
58      int ham = analogRead(pin);
59      int cikis;
60
```

```

61     if (ham > merkez + OLU_BOLGE) {
62         cikis = map(ham, merkez + OLU_BOLGE, 4095, 1500, 2000);
63     } else if (ham < merkez - OLU_BOLGE) {
64         cikis = map(ham, 0, merkez - OLU_BOLGE, 1000, 1500);
65     } else {
66         cikis = 1500;                                // olu bolge icinde: tam merkez
67     }
68
69     cikis = constrain(cikis, 1000, 2000);
70     if (ters) cikis = 3000 - cikis;                    // eksen tersine cevir
71     return (uint16_t)cikis;
72 }

```

Fonksiyon üç dallıdır: merkezin üstü, merkezin altı ve ölü bölge. İki ayrı map() çağırısı kullanılmasının nedeni, merkezin gerçek orta noktada olmamasıdır — tek bir ölçekleme, merkez kaçıklığını çıkışa taşırdı. ters parametresi, motor yerleşimine göre eksen yönünün yazılımda çevrilebilmesini sağlar; kablo değiştirmeye gerek bırakmaz.

3.4 Kilitli gaz mantığı

Bu blok, projenin en öğretici tasarım kararlarından birini içerir. Kumanda modülünde tek bir merkezleme yayı iki eksen birden merkeziyordu; gaz ekseninin bıraktığı yerde kalması için yay sökölünce yaw eksen de merkezlemesini kaybetti. Mekanik çözüm geri alındı ve sorun yazılım katmanında çözüldü.

kilitli gaz + karesel tepki · satır 90–112

```

90  /* ----- GAZ: kilitli mod, karesel tepki -----
91  Joystick modulunde tek merkezleme yayi iki eksen birden kontrol
92  ediyordu; yay sokulduygunde yaw eksen de merkezlemesini kaybetti.
93  Cozum: yay yerinde birakildi, gaz yazilimda kilitlendi.
94  kol yukari -> gaz artar    kol serbest -> gaz sabit kalir
95  Karesel egri sayesinde kucuk hareket hassas, tam hareket hizlidir. */
96  int hamGaz = analogRead(PIN_GAZ);
97  float oran = 0.0;
98
99  if (hamGaz > MERKEZ_GAZ + GAZ_OLU_BOLGE) {
100     oran = (float)(hamGaz - MERKEZ_GAZ - GAZ_OLU_BOLGE)
101           / (4095.0 - MERKEZ_GAZ - GAZ_OLU_BOLGE);
102     oran = constrain(oran, 0.0, 1.0);
103     gaz += GAZ_HIZI * oran * oran;
104 } else if (hamGaz < MERKEZ_GAZ - GAZ_OLU_BOLGE) {
105     oran = (float)(MERKEZ_GAZ - GAZ_OLU_BOLGE - hamGaz)
106           / (MERKEZ_GAZ - GAZ_OLU_BOLGE);
107     oran = constrain(oran, 0.0, 1.0);
108     gaz -= GAZ_HIZI * oran * oran;
109 }
110
111 gaz = constrain(gaz, 1000, 2000);
112 veri.kanal[0] = (uint16_t)gaz;

```

Kol itildiği sürece gaz değişkeni artar, serbest bırakıldığında son değerinde donar. **Karesel tepki eğrisi** (oran * oran) ise kolun az itildiği durumda değişimi çok yavaşlatır, tam itildiğinde hızlandırır. Bunun pratik karşılığı, aracın havada asılı kaldığı gaz noktası çevresinde ince ayar yapılabilmesidir — doğrusal bir eğride bu bölge fazla hassas olurdu.

Emniyet notu

gaz değişkeni setup() içinde her açılışta 1000'e sıfırlanır. Kilitli gaz mantığının kabul edilebilir olmasının koşulu budur: kumanda resetlense bile gaz sıfırdan başlar, önceki değeri hatırlamaz.

4. Alıcı Yazılımı — Uçuş Kontrolcüsü

Alıcı modülü sistemin çekirdeğidir. Telsiz alımı, atalet kestirimi, emniyet mantığı, kontrol yasası ve motor sürüşü tek bir sabit periyotlu döngüde birleşir. Aşağıda modül, mantıksal bloklara ayrılarak tam olarak sunulmuştur.

4.1 Motor numaralandırma ve pin haritası

pin tanımları · satır 27–50

```

27  /* ----- PINLER */
28  #define PIN_NRF_CE    4
29  #define PIN_NRF_CSN   5
30  //    NRF SCK 18 | MISO 19 | MOSI 23  (VSPI varsayılan)
31  #define PIN_I2C_SDA   21
32  #define PIN_I2C_SCL   22
33
34  // Motor numaralandırması - X konfigürasyonu, USTTEN bakış
35  //
36  //      ON (kırmızı kollar)
37  //      M4 \                / M1
38  //      (CCW)                (CW)
39  //      \      _____ /
40  //      |      |         |
41  //      |      |         |
42  //      /      \       \
43  //      (CW)                (CCW)
44  //      M3 /                \ M2
45  //      ARKA (siyah kollar)
46  //
47  #define PIN_M1    12  // on-sağ , CW
48  #define PIN_M2    13  // arka-sağ, CCW
49  #define PIN_M3    14  // arka-sol, CW
50  #define PIN_M4    27  // on-sol , CCW

```

Motor numaralandırması yorum bloğundaki şemayla belgelenmiştir. Bu belgeleme kozmetik değildir: karıştırma matrisinin işaretleri doğrudan bu yerleşime bağlıdır ve yanlış eşleştirme, aracın düzeltme yerine ters yönde tepki vermesine yol açar.

4.2 Sistem sabitleri

sabitler · satır 52–66

```

52  /* ----- SABİTLER */
53  const byte    ADRES[6]      = "DRN01";
54  const uint8_t RF_KANAL     = 100;    // 2.4 GHz WiFi bandından uzak
55  const uint16_t PWM_MIN     = 1000;   // us - motor durdu
56  const uint16_t PWM_MAX     = 2000;   // us - tam gaz
57  const uint16_t PWM_ARM     = 1100;   // us - rolanti (armed, yerde)
58  const uint16_t PWM_TAVAN   = 1900;   // us - PID için bas payı
59  const uint16_t GAZ_ESİK    = 1050;   // altında "gaz kapalı" sayılır
60  const uint16_t BAGLANTI_ZAMAN = 500; // ms - failsafe esigi
61  const float   DONGU_HZ     = 250.0;  // Hz - kontrol döngüsü
62  const uint32_t DONGU_US     = (uint32_t)(1000000.0 / DONGU_HZ);
63
64  // Kumanda komut ölçekleri
65  const float   MAX_ACI       = 25.0;   // derece - roll/pitch komut sınırı
66  const float   MAX_YAW_HIZ   = 120.0;  // derece/s - yaw komut sınırı

```

İki değer özellikle dikkat ister. PWM_TAVAN 1900 µs olarak seçilmiştir, 2000 değil: PID'nin düzeltme yapabilmesi için tepe gazın altında bir **baş payı** kalmalıdır. Tüm motorlar tavana dayanmışsa kontrolcü artık hiçbir eksenle düzeltme üretemez. DONGU_HZ ise 250 Hz'dir; bu değerden türetilen sabit periyot, PID'nin türev ve integral terimlerinin tutarlılığı için zorunludur.

4.3 Kazanç yapısı

PID kazançları · satır 68–77

```
68  /* ----- PID KAZANCLARI (AYAR) */
69  // Ayar sirasi: once Kp, sonra Kd, en son Ki. Detay: Tasarim Dok. Bolum 8.3
70  struct Kazanc { float kp, ki, kd; };
71
72  Kazanc G_ROLL = { 0.0f, 0.0f, 0.0f }; // baslangic: 1.30 / 0.00 / 12.0
73  Kazanc G_PITCH = { 0.0f, 0.0f, 0.0f }; // roll ile ayni alinir (simetrik)
74  Kazanc G_YAW = { 0.0f, 0.0f, 0.0f }; // baslangic: 2.50 / 0.00 / 0.0
75
76  const float I_SINIR = 120.0f; // integral windup siniri (us)
77  const float PID_SINIR = 320.0f; // eksen basina toplam cikis siniri (us)
```

Kazançlar bilinçli olarak sıfırdır

Modül, kazançlar sıfırken güvenli biçimde çalışır: motorlar yalnızca gaz komutunu izler, düzeltme uygulanmaz. Ayar prosedürü **SDD-QC-001 Bölüm 8.3**'te tanımlıdır ve test standı üzerinde, pervaneler takılı olarak yapılır. Yorum satırlarındaki değerler başlangıç noktası önerisidir, doğrulanmış değer değildir.

4.4 Veri yapıları ve durum değişkenleri

veri yapıları · satır 79–104

```
79  /* ----- GLOBAL NESNE */
80  RF24      radio(PIN_NRF_CE, PIN_NRF_CSN);
81  MPU6050   mpu(Wire);
82  Servo     motor[4];
83  const int MOTOR_PIN[4] = { PIN_M1, PIN_M2, PIN_M3, PIN_M4 };
84
85  /* ----- VERI YAPILARI */
86  struct Paket {
87      uint16_t kanal[4]; // telsiz uzerinden gelen cerceve (10 bayt)
88      uint16_t sayac;    // 0:gaz 1:yaw 2:pitch 3:roll (1000-2000)
89      // paket sirasi - kayip tespiti icin
90  };
91  Paket gelen;
92
93  uint16_t kanal[4] = { 1000, 1500, 1500, 1500 }; // guvenli baslangic
94  uint32_t sonPaket = 0;
95  uint16_t sonSayac = 0;
96  uint32_t kayipPaket = 0;
97  bool     baglanti = false;
98  bool     armed     = false;
99  uint32_t donguSon = 0;
100 float     dt        = 1.0f / DONGU_HZ;
101
102 struct PidDurum { float integral, oncekiHata; };
103 PidDurum dRoll = { 0, 0 };
104 PidDurum dPitch = { 0, 0 };
105 PidDurum dYaw = { 0, 0 };
```

Paket yapısı verici tarafındakiyle birebir aynıdır. Alan sırası veya tipi bir tarafta değişirse veri kayar ve anlamsız değerler okunur — bu, doğrudan bayt kopyalamaya dayanan protokollerin bilinen maliyetidir ve iki dosyanın birlikte sürümlemesini gerektirir.

4.5 Yardımcı fonksiyonlar

kanalKomut() ve pidHesapla() · satır 110–149

```
110 /* Kanal degerini (1000-2000) simetrik komut araligina donusturur.
111     1500 us -> 0.0 ; 2000 us -> +limit ; 1000 us -> -limit */
112 float kanalKomut(uint16_t us, float limit) {
113     float x = ((float)us - 1500.0f) / 500.0f; // -1.0 .. +1.0
114     x = constrain(x, -1.0f, 1.0f);
```

```

115     return x * limit;
116 }
117
118 /* Tek eksen PID. Turev, olculen degerden alinir (derivative-on-measurement)
119    - boylece kumanda komutu aniden degistiginde turev sicramasi olmaz. */
120 float pidHesapla(PidDurum &d, const Kazanc &g,
121                 float hedef, float olculen, bool bisikletle) {
122     float hata = hedef - olculen;
123
124     // Integral: sadece armed ve gaz acikken birikir, aksi halde sifirlanir
125     if (bisikletle) {
126         d.integral += g.ki * hata * dt;
127         d.integral = constrain(d.integral, -I_SINIR, I_SINIR);
128     } else {
129         d.integral = 0.0f;
130     }
131
132     float turev = (olculen - d.oncekiHata) / dt;    // olculen uzerinden
133     d.oncekiHata = olculen;
134
135     float cikis = g.kp * hata + d.integral - g.kd * turev;
136     return constrain(cikis, -PID_SINIR, PID_SINIR);
137 }
138
139 /* Tum PID durumlarini sifirlar - disarm ve failsafe aninda cagrilir */
140 void pidSifirla() {
141     dRoll = { 0, 0 };
142     dPitch = { 0, 0 };
143     dYaw = { 0, 0 };
144 }
145
146 /* Dort motora ayni degeri yazar */
147 void motorlariYaz(uint16_t us) {
148     for (int i = 0; i < 4; i++) motor[i].writeMicroseconds(us);
149 }

```

Türev terimi hata yerine ölçülen değerden alınır. Klasik PID'de türev, hatanın değişim hızıdır; ancak pilot komutu ani değiştiğinde hata da ani sıçrar ve türev terimi büyük bir darbe üretir (*derivative kick*). Ölçülen açıdan türev almak bu darbeyi tümüyle ortadan kaldırır, çünkü aracın fiziksel açısı ani sıçrayamaz. Bunun bedeli, komut değişimlerine tepkinin bir miktar yumuşamasıdır — uçuş kontrolünde kabul edilen bir takas.

Integral windup koruması iki katmanlıdır: birikim yalnızca bisikletle bayrağı doğruyken yapılır (silahlı ve gaz eşğin üstünde), ayrıca birikmiş değer $\pm 120 \mu s$ ile sınırlanır. Bu olmadan, araç yerde beklerken integral terimi sürekli birikir ve kalkış anında ani bir yatışa yol açardı (H-09).

4.6 Başlatma dizisi

setup() · satır 154–202

```

154 void setup() {
155     Serial.begin(115200);
156     delay(300);
157     Serial.println(F("\n=== ESP32 UCUS KONTROL v3.0 ==="));
158
159     /* --- 1. ADIM: motorlar once susturulur -----
160        ESC'ler enerji aldiginda gecerli bir dusuk sinyal gormezse hata bipi
161        calar veya beklenmedik sekilde donebilir. Bu yuzden ilk is motor
162        cikislarini PWM_MIN'e sabitlemektir. */
163     for (int i = 0; i < 4; i++) {
164         motor[i].setPeriodHertz(50);
165         motor[i].attach(MOTOR_PIN[i], PWM_MIN, PWM_MAX);
166         motor[i].writeMicroseconds(PWM_MIN);
167     }

```



```

168 Serial.println(F("[OK] Motor cikislari PWM_MIN'e sabitlendi"));
169
170 /* --- 2. ADIM: atalet olcum birimi ----- */
171 Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
172 Wire.setClock(400000); // hizli I2C - dongu butcesi icin
173 byte durum = mpu.begin();
174 if (durum != 0) {
175     Serial.print(F("[HATA] MPU6050 bulunamadi, kod: "));
176     Serial.println(durum);
177     while (1) { motorlariYaz(PWM_MIN); delay(200); } // guvenli kilit
178 }
179 Serial.println(F("[..] Kart DUZ ve HAREKETSIZ dursun, kalibre ediliyor"));
180 delay(1200);
181 mpu.calcOffsets(); // gyro + ivme sapma giderme
182 Serial.println(F("[OK] MPU6050 hazir"));
183
184 /* --- 3. ADIM: telsiz ----- */
185 if (!radio.begin()) {
186     Serial.println(F("[HATA] nRF24 bulunamadi"));
187     while (1) { motorlariYaz(PWM_MIN); delay(200); }
188 }
189 radio.openReadingPipe(0, ADRES);
190 radio.setPALevel(RF24_PA_LOW); // ucusta RF24_PA_MAX yapilir
191 radio.setDataRate(RF24_250KBPS); // dusuk hiz = uzun menzil
192 radio.setChannel(RF_KANAL);
193 radio.startListening();
194 Serial.println(F("[OK] nRF24 dinlemede"));
195
196 sonPaket = millis();
197 donguSon = micros();
198 Serial.println(F("=== HAZIR - arming icin gaz kolunu en alta cekin ===\n"));
199 }
200
201 /* ===== */
202 /* ANA DONGU - sabit 250 Hz */

```

Başlatma sırası emniyet gerekçesiyle sabittir. **İlk iş motor çıkışlarını asgari değere sabitlemektir**; ESC'ler enerji aldıklarında geçerli bir düşük sinyal görmezlerse hata durumuna geçer veya beklenmedik biçimde dönebilir. Ancak bundan sonra sensör ve telsiz başlatılır.

IMU veya telsiz başlatılamazsa yazılım sonsuz döngüye girer ve bu döngü içinde motorları asgari değerde tutmayı sürdürür. Bu, sessizce hatalı çalışmaktansa **güvenli biçimde durmayı** tercih eden bir tasarımdır (SYS-S-05).

4.7 Ana döngü — alım, failsafe ve arming

loop() – 1. bölüm • satır 204–252

```

204 void loop() {
205
206     /* ----- 1. TELSİZ: gelen cerceveyi al ve dogrula ----- */
207     if (radio.available()) {
208         radio.read(&gelen, sizeof(gelen));
209
210         // Aralik disi deger = bozuk cerceve. Bu filtre olmadan tek bir hatali
211         // paket motorlara sicrama komutu olarak gidebilir.
212         bool gecerli = true;
213         for (int i = 0; i < 4; i++)
214             if (gelen.kanal[i] < 950 || gelen.kanal[i] > 2050) gecerli = false;
215
216         if (gecerli) {
217             // paket kaybi istatistigi (telemetry / menzil analizi icin)
218             if (sonSayac != 0 && (uint16_t)(gelen.sayac - sonSayac) > 1)
219                 kayipPaket += (uint16_t)(gelen.sayac - sonSayac) - 1;
220             sonSayac = gelen.sayac;

```

```

221
222     for (int i = 0; i < 4; i++) kanal[i] = gelen.kanal[i];
223     sonPaket = millis();
224     baglanti = true;
225 }
226 }
227
228 /* ----- 2. FAILSAFE: baglanti kaybi ----- */
229 if (millis() - sonPaket > BAGLANTI_ZAMAN) {
230     if (baglanti) Serial.println(F(">>> FAILSAFE - baglanti kesildi"));
231     baglanti = false;
232     armed = false;           // silahsızlan
233     kanal[0] = PWM_MIN;      // gaz kesildi
234     kanal[1] = kanal[2] = kanal[3] = 1500;
235     pidSifirla();
236     motorlariYaz(PWM_MIN);
237 }
238
239 /* ----- 3. ARMING MANTIGI ----- */
240 Silahlanma sarti: baglanti VAR + gaz kolu en altta.
241 Silahsızlanma : gaz kolu esigin altina inince.
242 Bu, kod resetlendiğinde veya kumanda yeniden bağlandığında motorların
243 kendiliğinden çalışmasını engelleyen birincil emniyet katmanıdır. */
244 if (!armed && baglanti && kanal[0] <= GAZ_ESIK) {
245     armed = true;
246     pidSifirla();
247     Serial.println(F(">>> ARMED - motorlar aktif"));
248 }
249 if (armed && kanal[0] <= GAZ_ESIK) {
250     // yerde bekleme: PID birikmesin
251     pidSifirla();
252 }

```

Emniyet blokları zamanlama kapısının **öncesinde** yer alır. Bunun anlamı, failsafe ve arming kontrollerinin her turda — periyot dolmasa bile — çalışmasıdır. Kontrol yasası 4 ms'de bir güncellenir, ancak bağlantı kaybı algılaması mümkün olan en kısa sürede yapılır.

Failsafe tetiklendiğinde yalnızca gaz kesilmez; sistem aynı zamanda armed bayrağını düşürür ve PID durumlarını sıfırlar. Bağlantı geri geldiğinde motorlar kendiliğinden çalışmaz — pilotun silahlanma dizisini yeniden uygulaması gerekir.

4.8 Ana döngü — kestirim, PID ve motor karıştırma

loop() – 2. bölüm · satır 254–295

```

254 /* ----- 4. SABIT DONGU ZAMANLAMASI ----- */
255 PID turev ve integral terimleri sabit dt varsayar. Dongu suresi
256 degiskense kazanclar da fiilen degisir ve ayar tutmaz. */
257 uint32_t simdi = micros();
258 if (simdi - donguSon < DONGU_US) return;
259 dt = (simdi - donguSon) / 1000000.0f;
260 donguSon = simdi;
261
262 /* ----- 5. ATALET OLCUMU ----- */
263 mpu.update();
264 float rollOlculen = mpu.getAngleX(); // derece
265 float pitchOlculen = mpu.getAngleY(); // derece
266 float yawHizi = mpu.getGyroZ(); // derece/s (aci degil!)
267
268 /* ----- 6. KUMANDA KOMUTLARI ----- */
269 float rollHedef = kanalKomut(kanal[3], MAX_ACI);
270 float pitchHedef = kanalKomut(kanal[2], MAX_ACI);
271 float yawHedef = kanalKomut(kanal[1], MAX_YAW_HIZ);
272 uint16_t gaz = constrain(kanal[0], PWM_MIN, PWM_TAVAN);
273

```

```

274  /* ----- 7. PID ----- */
275  Roll ve pitch : ACI modu - hedef aci, olculen aci
276  Yaw           : HIZ modu - hedef donus hizi, olculen gyro hizi
277  Yaw'da aci kullanilmaz cunku manyetometre olmadan yaw acisi surekli
278  kayar (drift). Donus hizi ise driftten etkilenmez. */
279  bool integralAktif = armed && (gaz > GAZ_ESIK + 60);
280
281  float uRoll = pidHesapla(dRoll, G_ROLL, rollHedef, rollOlculen, integralAktif);
282  float uPitch = pidHesapla(dPitch, G_PITCH, pitchHedef, pitchOlculen, integralAktif);
283  float uYaw = pidHesapla(dYaw, G_YAW, yawHedef, yawHizi, integralAktif);
284
285  /* ----- 8. MOTOR KARISTIRMA (X konfigurasyonu) ----- */
286  Isaret mantigi:
287  roll olculen > hedef (saga fazla yatik) -> sag motorlar artar
288  pitch olculen > hedef (burun fazla yukari) -> on motorlar artar
289  yaw saga donus komutu -> CCW motorlar artar (tepki torku)
290  M1 on-sag CW | M2 arka-sag CCW | M3 arka-sol CW | M4 on-sol CCW */
291  int m[4];
292  m[0] = gaz - uRoll - uPitch - uYaw; // M1 on-sag (CW)
293  m[1] = gaz - uRoll + uPitch + uYaw; // M2 arka-sag (CCW)
294  m[2] = gaz + uRoll + uPitch - uYaw; // M3 arka-sol (CW)
295  m[3] = gaz + uRoll - uPitch + uYaw; // M4 on-sol (CCW)

```

Yaw eksenini için `getAngleZ()` değil `getGyroZ()` kullanılır. Gerekçe SDD-QC-001 Bölüm 3.3'te ayrıntılıdır: manyetometre yokluğunda yaw açısı sürekli sürüklenir, ancak açısal hız ölçümü bu sürüklenmeden etkilenmez. Kontrol değişkeninin doğru seçilmesi, donanım eksikliğini yazılım mimarisiyle telafi etmenin örneğidir.

Motor karıştırma bloğundaki işaretler, motorun gövde üzerindeki konumundan ve dönüş yönünden türetilir. Yaw düzeltmesi, saat yönünün tersine dönen motorlarla saat yönünde dönenler arasındaki tepki torku farkı üzerinden üretilir; bu nedenle işaretler çapraz motorlarda eşleşir.

4.9 Çıkış sınırlama ve telemetri

loop() – 3. bölüm · satır 297–322

```

297  /* ----- 9. ÇIKIŞ SINIRLAMA VE YAZMA ----- */
298  if (!armed || gaz <= GAZ_ESIK) {
299  // Silahsız veya gaz kapalı: PID ne derse desin motorlar rolantide.
300  uint16_t bekleme = armed ? PWM_MIN : PWM_MAX;
301  motorlariYaz(bekleme);
302  } else {
303  for (int i = 0; i < 4; i++) {
304  m[i] = constrain(m[i], PWM_ARM, PWM_MAX);
305  motor[i].writeMicroseconds((uint16_t)m[i]);
306  }
307  }
308
309  /* ----- 10. TELEMETRI (10 Hz) ----- */
310  static uint32_t yaz = 0;
311  if (millis() - yaz > 100) {
312  yaz = millis();
313  Serial.print(baglanti ? F("OK ") : F("KOP "));
314  Serial.print(armed ? F("ARM ") : F("--- "));
315  Serial.print(F("T:")); Serial.print(kanal[0]);
316  Serial.print(F(" R:")); Serial.print(rollOlculen, 1);
317  Serial.print(F(" P:")); Serial.print(pitchOlculen, 1);
318  Serial.print(F(" Yr:")); Serial.print(yawHizi, 1);
319  Serial.print(F(" | M "));
320  for (int i = 0; i < 4; i++) { Serial.print(m[i]); Serial.print(' '); }
321  Serial.print(F("| kayip:")); Serial.println(kayipPaket);
322  }

```

Son emniyet kapısı buradadır: silahlı değilse veya gaz kapalıysa, PID'nin ürettiği değer ne olursa olsun motorlara asgari komut yazılır. Bu, kontrol yasasındaki olası bir hatanın motorları çalıştırmasını engelleyen bağımsız bir katmandır — savunma yazılımlarındaki *defence in depth* ilkesinin küçük ölçekli uygulaması.

Telemetri çıktısı 10 Hz'e seyreltilmiştir. Seri port yazma işlemi görece pahalıdır ($\sim 800 \mu s$); her döngüde çalıştırılıyorsa 4 ms'lik bütçenin beşte birini tek başına tüketirdi.

5. Araç Yazılımları

Araç yazılımları, uçuş yazılımına geçmeden önce donanımı doğrulamak için yazılmıştır. Küçük olmaları aldaticıdır — her ikisi de geri döndürülmesi pahalı hataları önler.

5.1 ESC uç nokta kalibrasyonu

Kalibrasyon yapılmazsa her ESC kendi fabrika gaz aralığını kullanır. Aynı PWM değeri farklı motorlarda farklı devir üretir; araç havada sürekli bir yöne kayar ve bu, PID ayarıyla düzeltilemeyecek bir asimetridir.

arac_esc_kalibrasyon.ino · satır 19–54

```

19  #include <ESP32Servo.h>
20
21  Servo esc[4];
22  const int PIN[4] = { 12, 13, 14, 27 };
23
24  void hepsineYaz(uint16_t us) {
25      for (int i = 0; i < 4; i++) esc[i].writeMicroseconds(us);
26  }
27
28  void setup() {
29      Serial.begin(115200);
30      delay(500);
31
32      for (int i = 0; i < 4; i++) {
33          esc[i].setPeriodHertz(50);
34          esc[i].attach(PIN[i], 1000, 2000);
35      }
36
37      hepsineYaz(2000);
38      Serial.println(F("=== MAX sinyal aktif (2000 us) ==="));
39      Serial.println(F("SIMDI bataryayı tak. Bip seslerini bekle."));
40      delay(10000);
41
42      hepsineYaz(1000);
43      Serial.println(F("=== MIN sinyal aktif (1000 us) ==="));
44      Serial.println(F("Onay bipleri gelmeli."));
45      delay(5000);
46
47      Serial.println(F("KALIBRASYON BITTI."));
48  }
49
50  void loop() {
51      // Sinyal sürdürülür. Kesilirse ESC "sinyal yok" hata bipi calar.
52      hepsineYaz(1000);
53      delay(20);
54  }
```

loop() neden boş değil

İlk sürümde loop() boş bırakılmıştı. Kalibrasyon bittiğinde sinyal üretimi durdu ve ESC'ler sürekli hata bipi çalmaya başladı — sinyal kaybı, ESC için bir arıza durumudur. Düzeltme olarak döngü, asgari sinyali sürdürmeye devam eder. Küçük bir ayrıntı gibi görünse de, ESC'nin sinyal beklentisinin sürekli olduğunu anlamayı gerektirir.

5.2 Motor dönüş yönü testi

X konfigürasyonunda çapraz motorlar aynı yöne, komşu motorlar ters yöne dönmelidir. Bu düzen sağlanmazsa dört motorun tepki torku toplamı sıfırlanmaz ve araç yaw ekseninde kontrolsüz döner.

arac_motor_test.ino · satır 19–59

```

19  #include <ESP32Servo.h>
```

```

20
21 Servo esc[4];
22 const int PIN[4] = { 12, 13, 14, 27 };
23 const uint16_t TEST_GAZ = 1150;    // rolanti uzeri - yon gormeye yeter
24
25 void setup() {
26     Serial.begin(115200);
27     delay(500);
28
29     for (int i = 0; i < 4; i++) {
30         esc[i].setPeriodHertz(50);
31         esc[i].attach(PIN[i], 1000, 2000);
32         esc[i].writeMicroseconds(1000);
33     }
34
35     Serial.println(F("=== MOTOR TESTI ==="));
36     Serial.println(F("1-4 : o motoru 3 saniye dondur"));
37     Serial.println(F("0   : hepsini durdur"));
38     Serial.println(F("PERVANE TAKILI OLMAYACAK"));
39 }
40
41 void loop() {
42     if (!Serial.available()) return;
43     char c = Serial.read();
44
45     if (c >= '1' && c <= '4') {
46         int m = c - '1';
47         Serial.print(F(">> Motor ")); Serial.print(m + 1);
48         Serial.println(F(" donuyor (3 sn)"));
49         esc[m].writeMicroseconds(TEST_GAZ);
50         delay(3000);
51         esc[m].writeMicroseconds(1000);
52         Serial.println(F("   durdu"));
53     }
54
55     if (c == '0') {
56         for (int i = 0; i < 4; i++) esc[i].writeMicroseconds(1000);
57         Serial.println(F(">> HEPSI DURDU"));
58     }
59 }

```

Test kodu, motorları **tek tek** çalıştırır. Bu, tanı koymayı mümkün kılan tasarım tercihidir: dördü aynı anda çalıştırsaydı yanlış dönen motoru ayırt etmek zorlaşırdı. Devreye alma sırasında her motor önce geçici bağlantıyla test edilmiş, dönüş yönü doğrulandıktan sonra kalıcı olarak lehimlenmiştir — geri döndürülemez adımın doğrulamadan sonraya bırakılması ilkesi.

6. Emniyet Mekanizmalarının Özeti

Emniyet, tek bir mekanizmaya değil birbirinden bağımsız katmanlara dayanır. Aşağıdaki tablo, her katmanın hangi kod bloğunda gerçekleştiğini ve hangi arıza modunu (SDD-QC-001 Bölüm 6) karşıladığını gösterir.

#	Mekanizma	Kod konumu	Karşıladığı risk
1	Başlangıçta motor çıkışlarının asgariye sabitlenmesi	setup(), 1. adım	H-03
2	IMU / telsiz başlatma hatasında güvenli kilit	setup(), 2.-3. adım	H-06
3	Çerçeve aralık süzgeci (950–2050 μ s)	loop(), 1. blok	H-02
4	500 ms zaman aşımli failsafe	loop(), 2. blok	H-01
5	Failsafe'te otomatik silahsızlanma	loop(), 2. blok	H-01
6	Arming zorunluluğu (gaz asgari şartı)	loop(), 3. blok	H-03
7	Koşullu integral birikimi + windup sınırı	pidHesapla()	H-09
8	Eksen başına PID çıkış sınırlaması (± 320 μ s)	pidHesapla()	—
9	Gaz tavanı (1900 μ s) ile düzeltme baş payı	loop(), 6. blok	—
10	Silahsız / gaz kapalı durumunda koşulsuz asgari çıkış	loop(), 9. blok	H-03

Tasarım ilkesi

10 numaralı mekanizma bilinçli bir fazlalıktır: 6 numaralı arming kontrolü zaten aynı işi yapar. Ancak emniyet-kritik yollarda tek bir kontrole güvenmek, o kontroldeki bir mantık hatasının doğrudan tehlikeye dönüşmesi anlamına gelir. Bağımsız iki kapı, birinin hatalı olması hâlinde diğ erinin tutmasını sağlar.

7. Zamanlama Bütçesi ve Kaynak Kullanımı

Kontrol döngüsünün 4 ms'lik periyodu içinde yapılan işlerin süreleri aşağıdadır. Ölçümler mikrosaniye sayacıyla alınmış yaklaşık değerlerdir.

İşlem	Süre	Bütçe payı	Not
nRF24 çerçeve okuma (SPI)	~120 µs	%3.0	Yalnızca çerçeve geldiğinde
MPU6050 okuma + açış hesabı (I ² C)	~900 µs	%22.5	En büyük tekil kalem
PID hesabı (3 eksen)	~40 µs	%1.0	Kayan nokta birimi kullanılıyor
Motor karıştırma + sınırlama	< 20 µs	%0.5	Tamsayı aritmetiği
PWM yazma (4 kanal, LEDC)	~30 µs	%0.8	Donanım hızlandırılmalı
Telemetri (10 Hz seyriltilmiş)	~800 µs	%2.0	Etkin pay: 800/10 turda
Toplam (tipik)	≈ 1.2 ms	%30	Yaklaşık %70 rezerv

I²C hızı 400 kHz'e çıkarılarak IMU okuma süresi yaklaşık yarıya indirilmiştir; varsayılan 100 kHz ile bu kalem tek başına bütçenin yarısına yaklaşıyordu. Kalan rezerv, ileride eklenecek irtifa tutma, telemetri geri kanalı veya kara kutu kaydı gibi işlevler için bilinçli olarak ayrılmıştır.

7.1 Çift çekirdek kullanımı

Mevcut sürüm tek çekirdek üzerinde çalışır. ESP32'nin ikinci çekirdeği, ileride telemetri ve haberleşme görevlerine ayrılarak kontrol döngüsünün determinizmi daha da güçlendirilebilir. Bu, FreeRTOS görev ayrımı gerektiren orta vadeli bir geliştirmedir.

8. Derleme, Bağımlılıklar ve Devreye Alma

8.1 Araç zinciri

Bileşen	Sürüm / ayar	Not
Arduino IDE	2.x	—
ESP32 kart paketi	esp32 by Espressif Systems 3.3.x	Boards Manager üzerinden
Kart seçimi	ESP32 Dev Module	—
Yükleme hızı	115200	921600'de seri gürültü hatası gözlemlendi
USB köprü sürücüsü	Silicon Labs CP210x VCP	Sürücü kurulmadan port görünmez
Seri monitör	115200 baud	—

8.2 Kütüphane bağımlılıkları

Kütüphane	Yazar	Kullanan modül	İşlev
RF24	TMRh20	verici, alıcı	nRF24L01+ sürücüsü
MPU6050_light	rfetick	alıcı	IMU okuma ve açı kestirimi
ESP32Servo	K. Harrington, J. Bennett	alıcı, araç kodları	ESC için servo darbesi üretimi
Wire	çekirdek	alıcı	I ² C
SPI	çekirdek	verici, alıcı	SPI

Kütüphane seçiminde isim benzerliğine dikkat edilmelidir: RF24 adıyla birden fazla paket vardır ve yalnızca TMRh20 sürümü bu kodla uyumludur. Aynı biçimde Servo kütüphanesi AVR mimarisi içindir, ESP32'de derlenmez.

8.3 Devreye alma sırası

- Batarya çıkarılır.** Kod yükleme sırasında ESC'nin BEC hattı ile USB beslemesi çakışır ve seri iletişim bozulur (D-02).
- arac_esc_kalibrasyon yüklenir, seri monitör açılır.
- "MAX sinyal aktif" mesajı görüldüğünde batarya takılır; bip dizisi beklenir.
- arac_motor_test yüklenir; motorlar tek tek çalıştırılıp dönüş yönleri doğrulanır.
- Yönler doğrulandıktan sonra motor kabloları kalıcı olarak lehimlenir.
- verici_v2_kumanda verici karta, alıcı_v3_ucus_kontrol alıcı karta yüklenir.
- Kazançlar sıfırken bağlantı, arming ve failsafe davranışı doğrulanır — **pervanesiz**.
- Test standına geçilir; işaret doğrulaması ve kazanç ayarı yapılır.

Kalıcı kural

Her kod yüklemesinden önce batarya çıkarılır. Bu, hem seri iletişim kararlılığı hem de yükleme sırasında motorların beklenmedik biçimde çalışmaması için gereklidir.

9. Sürüm Geçmişi

Sürüm	Modül	Değişiklik	Gerekçe
v1.0	verici	Temel 4 kanal aktarımı, doğrusal ölçekleme	İlk çalışan haberleşme
v1.1	verici	Merkez kalibrasyonu ve ± 60 ölü bölge eklendi	Ham ADC merkezi 2048 değil
v2.0	verici	Kilitli gaz + karesel tepki eğrisi	Paylaşımlı merkezleme yayı kısıtı (D-01)
v2.1	verici	Ölü bölge ± 80 'e genişletildi	Yaw kanalında artık titreşim ölçüldü
v1.0	alıcı	Telsiz alımı ve seri çıktı	Bağlantı doğrulaması
v1.1	alıcı	Aralık süzgeci ve failsafe eklendi	Boş tampon okuma arızası (D-03)
v2.0	alıcı	MPU6050 entegrasyonu, açılı çıktı	Kestirim katmanı
v3.0	alıcı	Arming, PID, motor karıştırma, ESC çıkışı, telemetri	Tam uçuş kontrol yazılımı

Çalışan her sürüm ayrı dosya adıyla saklanmıştır. Bu, hatalı bir değişiklikten geri dönüşü mümkün kılan en basit sürüm kontrolü biçimidir ve donanımla çalışırken özellikle değerlidir: yazılım hatası ile donanım arızasını ayırt etmenin en hızlı yolu, bilinen çalışan bir sürüme dönmektir.

Doküman sonu

Sistem düzeyi gereksinimler, mimari kararlar, tehlike analizi ve test kayıtları için **SDD-QC-001 — Sistem Tasarım Dokümanı**'na başvurulmalıdır.